exo 6. problem 1 solution

We consider a tree where every node has n children and each child has weighe 1/n of its parent.

1. General class for the object

we define a node in the tree.

Each node has a weighe and a list of children.

pseudocode

```
class Node
    weight
    children list
function Node(w)
weight = w
children = empty list
function add_child(child)
add child to children.
```

2. Generate a tree of depth $N = 3$

We build the tree recursively

Each node creates n children and each child weight is parent_weight/n.

Pseudocode

```
function GenerateTree(depth, n, weight)
  Create node with weight
  if depth == 0
  return node
repeat n times
  child = GenerateTree(depth-1, n, weight/n)
  add child to node.children
  return node.
```

Example:

Root = GenerateTree(3, n, 1)

3. Depth First Search (recursive)

Depth First Search visits one branch of the tree first and sums all weighes.

Pseudo code

```
function DFS (node)
sum = node.weight.
for each child in node.children
sum = sum + DFS (child)
return sum
```

This should return approximately 1 for different values of n.

4. Breadth First Search

Breadth Fist Search visits nodes level by level using a queue.

Pseudocode

```
function BFS (root)
create empty queue
enqueue root
sum ← 0
while queue not empty
node = dequeue
sum = sum + node.weight.
for each child in node.children
    ~~eq~~ enqueue child
return sum
```

The result should also be 1.

5. Flip the sign when visiting nodes

Each time we visit a node we change the sign of the weight.

Example:

+ becomes −

− becomes +

pseudocode.

```
function DFS_Flip(node)
node.weight = -node.weight
sum = node.weight
for each child in node.children
  sum = sum + DFS_Flip(child)
return sum
```

After running the search:

first run → result is 1

second run → result is −1

6. Recursive and non-recursive BFS

Non-recursive BFS

```
function BFS(root)
queue = new queue.
enqueue root
while queue not empty
    node = dequeue
    visit node
    for each child
    enqueue child
```

Recursive BFS

```
function BFS_recursive(queue)
if queue empty
    return
node = dequeue
visit node
add children to queue
call BFS_recursive(queue)
```

7. why BFS recursion is not recommended

Depth First search works well with recursion because recursion uses a stack, which mathes the DFS process.

Breadth First search needs a queue to visit nodes level by level.

Even if we write BFS recursively, we still need a queue, therefore recursion does not give any advantage and may cause stack overflow for large trees.

Because of this, BFS is usually implemented iteratively using a queue.